

交通大数据

Python 高阶算法库

顾子渊， 副研究员、博导
复杂交通网络研究中心
ziyuangu@seu.edu.cn



Python 高阶算法库

□ 学习目标

- 了解 Python 模块安装与基本用法
- 认识 Python 标准库
- 学习 Numpy 基本用法
- 学习 Pandas 基本用法
- 了解 Matplotlib、Scikit-Learn、TensorFlow



Python 模块安装与基本用法

Python 最大优势之一：强大的模块功能

□ 模块的使用方法

模块检索：

PyPI (<https://pypi.org/>) 是Python 官方的模块发布网站，采用输入**关键词**搜索并获取 Python 模块

模块安装：

① 使用 pip 安装：Windows系统下，在命令行中输入 “pip install 模块名” 即可

```
C:\WINDOWS\system32\cmd.exe
Microsoft Windows [版本 10.0.17763.503]
(c) 2018 Microsoft Corporation。保留所有权利。
C:\Users\yuany>pip install pandas
```

检查 Python 及 pip 是否添加至环境变量！



Python 模块安装与基本用法

Python 最大优势之一：强大的模块功能

□ 模块的使用方法

模块安装：

② Anaconda环境下：在 Anaconda Prompt 中输入 “conda install 模块名” 即可

```
Administrator: Anaconda Prompt (Anaconda3) - conda install scikit-learn
(environment1) C:\WINDOWS\system32>conda install scikit-learn
Collecting package metadata (current_repodata.json): done
Solving environment: done

## Package Plan ##

  environment location: C:\ProgramData\Anaconda3\envs\environment1

added / updated specs:
  - scikit-learn
```



Python 模块安装与基本用法

Python 最大优势之一：强大的模块功能

□ 模块的使用方法

模块安装：

- ③ 手动安装 whl 文件：手动下载 whl 文件，在存放文件的目录下输入 cmd，在命令行中输入“ pip install 文件名” 即可

Archived: Unofficial Windows Binaries for Python Extension Packages

by Christoph Gohlke. Updated on 26 June 2022 at 07:27 UTC.

This page provides 32 and 64-bit Windows binaries of many scientific open-source extension packages for the official [CPython distribution](#) of the [Python](#) programming language. A few binaries are available for the [PyPy distribution](#).

The files are unofficial (meaning: informal, unrecognized, personal, unsupported, no warranty, no liability, provided "as is") and made available for testing and evaluation purposes.

Most binaries are built from source code found on [PyPI](#) or in the projects public revision control systems. Source code changes, if any, have been submitted to the project maintainers or are included in the packages.

Refer to the documentation of the individual packages for license restrictions and dependencies.

<https://www.lfd.uci.edu/~gohlke/pythonlibs/>



Python 模块安装与基本用法

Python 最大优势之一：强大的模块功能

□ 模块的使用方法

模块使用：

以 pandas 为例：①导入 pandas 模块，设置别名为 pd，使用别名 pd 引用 read_csv()函数，读取文件名为 ' file.csv ' 的文件

```
1. import pandas as pd
2. pd.read_csv( 'file.csv' )
```

②也可以直接导入 read_csv() 函数并读取文件

```
1. from pandas import read_csv
2. read_csv( 'file.csv' )
```



认识 Python 标准库

□ Python 标准库

math 模块

Python 中用于科学计算的内置模块，其中的函数都经过了特殊的算法优化，效率较高

常用函数：

- ① 常规数学运算函数
- ② 幂函数与对数函数
- ③ 三角函数
- ④ 常用常数

1.	math.pi	#圆周率 π
2.	math.e	#e
3.	math.inf	#无穷大
4.	math.nan	#空值

1.	import math	#调用
2.	math.ceil(x)	#向上取整
3.	math.floor(x)	#向下取整
4.	math.fabs(x)	#求绝对值

1.	math.exp(x)	#返回 e^x
2.	math.log(x, base)	#返回自然对数 $\log(x)$, base 为可选参数, 若提供 base, 则返回 $\log(x)/\log(\text{base})$
3.	math.pow(x, y)	#返回 x^y
4.	math.sqrt(x)	#返回 x 的平方根



认识 Python 标准库

□ Python 标准库

time 模块

Python 中用于处理时间的一个内建模块

相关概念

- ① 时间戳：从1970年1月1日00:00:00开始按秒计算的偏移量
- ② 元组 (struct_time)：共有9个元素

属性	值
tm_year	年(例如2019)
tm_mon	月(1-12)
tm_mday	日(1-31)
tm_hour	时(0-23)
tm_min	分(0-59)
tm_sec	秒(0-6)
tm_wday	周几(0-6)0是周日
tm_yday	一年中第几天(1-366)
tm_isdst	是否夏令时(默认-1)



认识 Python 标准库

□ Python 标准库

time 模块

Python 中用于处理时间的一个内建模块

相关概念

- ① 时间戳：从1970年1月1日00:00:00开始按秒计算的偏移量
- ② 元组 (struct_time)：共有9个元素
- ③ 数据处理中经常遇到，将时间戳或字符串形式的时间转化为标准时间，方便进行日期的运算



认

Py

time

Python

相关概

①

②

③

```

1. import time
2. start = time.time()      #记录代码开始运行时间, time.time()用于获取当前时间戳
3. a = 1566897866          #类型1:时间戳数据,常见的的时间输入格式
4. c = time.localtime(a)   #将时间戳转换为时间格式
5. print(c)
6. #输出结果如下:
   time.struct_time(tm_year=2019, tm_mon=8, tm_mday=27, tm_hour=17, tm_min=24, tm_sec=26,
   tm_wday=1, tm_yday=239, tm_isdst=0)
7. a = "2017-6-11 17:51:30" #类型2:字符型时间数据
8. c=time.strptime(a,"%Y-%m-%d %H:%M:%S")
9. print(c)
10. #输出结果如下:
11. #time.struct_time(tm_year=2017, tm_mon=6, tm_mday=11, tm_hour=17, tm_min=51, tm_sec
   =30, tm_wday=6, tm_yday=162, tm_isdst=-1)
12. c.tm_year              #年
13. c.tm_mon               #月
14. c.tm_mday              #日
15. c.tm_hour              #小时
16. c.tm_min               #分钟
17. c.tm_sec               #秒
18. c.tm_wday              #星期几
19. c.tm_yday              #到当年1月1日的天数
20. end = time.time()      #运行结束时间
21. timeSpent = end - start #计算运行时间
22. print("Time spent: {0} s".format(timeSpent))

```

时间戳转换

字符串转换

struct_time

间, 方便



认识 Python 标准库

□ Python 标准库

random 模块

Python 内建的用于生成随机数的模块

常用函数

1. `import random`
2. `random.random()` #生成 0-1 之间均匀分布的随机浮点数
3. `random.normalvariate(0, 1)` #生成 1 个符合均值为 0, 方差为 1 正态分布的随机数
4. `[random.normalvariate(0, 1) for x in range(10)]` #生成长度为 10 的正态分布序列
5. `random.uniform(a, b)` #生成 [a, b] 区间内的随机浮点数
6. `random.randint(a, b)` #生成 [a, b] 区间内的随机整数
7. `random.choice(s)` #从序列 s 中随机获取一个值
8. `random.shuffle(s)` #将序列 s 中元素打乱
9. `random.sample(s, k)` #从序列 s 中获取长度为 k 的片段



认识 Python 标准库

□ numpy

numpy 是一个用于**科学计算**的模块，提供了高性能的**数组对象**、**矩阵对象**，以及众多基础数学、线性代数、基本统计运算等所需的相关函数

相关概念

- ① ndarray 对象：numpy 中一种最基本的数组对象，其大小固定，**内部数据类型需要一致**，在数学运算的效率、稳定性上均有较大优势

```
1. import numpy as np
2. a = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]]) #定义一个ndarray 对象
3. #[[1, 2, 3],
4. #[4, 5, 6],
5. #[7, 8, 9]]
```

数组形状

6. a.ndim	#ndarray 的维数(阶),输出为 2
7. a.shape	#ndarray 的形状,输出为(3, 3),对应(行数,列数)
8. a.size	#ndarray 的元素个数,输出为 9



认识 Python 标准库

□ numpy

numpy 是一个用于**科学计算**的模块，提供了高性能的**数组对象**、**矩阵对象**，以及众多基础数学、线性代数、基本统计运算等所需的相关函数

相关概念

- ① ndarray 对象：numpy 中一种最基本的数组对象，其大小固定，**内部数据类型需要一致**，在数学运算的效率、稳定性上均有较大优势
- ② ndarray 复制：有三种形式，分别为引用、浅复制、深复制

```
1.  b = a                #引用
2.  print(id(a)== id(b)) #id()能够输出对象的内存地址,输出为 True
3.  c = a.view()         #浅复制
4.  print(id(a)==id(c))  #输出为 False
5.  d = a.copy()         #深复制
```

修改原数组，新数组也会改变！！！！



认识 Python 标准库

□ numpy

numpy 是一个用于**科学计算**的模块，提供了高性能的**数组对象**、**矩阵对象**，以及众多基础数学、线性代数、基本统计运算等所需的相关函数

相关概念

- ① ndarray 对象：numpy 中一种最基本的数组对象，其大小固定，**内部数据类型需要一致**，在数学运算的效率、稳定性上均有较大优势
- ② ndarray 复制：有三种形式，分别为引用、浅复制、深复制
- ③ ndarray 下标存取：与内置的 list 对象的存取和切片方法类似



认识 Python 标准库

□ numpy

1.	<code>a[0, 1]</code>	#输出 2 (行、列的索引都是由 0 开始)
2.	<code>a[[0, 2], [1, 2]]</code>	#输出为 <code>array([2, 9])</code> , 等价于 <code>np.array([a[0, 1], a[2, 2]])</code>
3.	<code>a[:2, 1:3]</code>	#第 0 至 1 行, 第 1 至 2 列 <code>array([[2, 3], [5, 6]])</code>
4.	<code>a[2, 1:]</code>	#第 2 行, 第 1 至最后 1 列
5.	<code>a[:-1]</code>	#第 0 至倒数第 2 行
6.	<code>a[1, :]</code>	#第 1 行, shape 为 (3,), 一维, <code>array([4, 5, 6])</code>
7.	<code>a[1:2, :]</code>	#第 1 行, shape 为 (1, 3), 二维, <code>array([[4, 5, 6]])</code>
8.	<code>a[:, 1]</code>	#第 1 列, shape 为 (3,), 一维, <code>array([2, 5, 8])</code>
9.	<code>a[:, 1:2]</code>	#第 1 列, shape 为 (3, 1), 二维, <code>array([[2], [5], [8]])</code>

数值索引

1.	<code>a>2</code>	#输出为 <code>array([[False, False, True],</code>
2.		<code>[True, True, True],</code>
3.		<code>[True, True, True]])</code>
4.	<code>a[a>2]</code>	#输出数组中大于 2 的值, 为一个一维数组
5.	<code>a[(a>2) & (a<6)]</code>	#使用逻辑运算符连接多个条件, 注意不能使用 <code>and</code> , <code>or</code> 之类的关键词进行连接, 否则会报错

布尔值索引



认识 Python 标准库

□ numpy

numpy 是一个用于**科学计算**的模块，提供了高性能的**数组对象**、**矩阵对象**，以及众多基础数学、线性代数、基本统计运算等所需的相关函数

相关概念

- ① ndarray 对象：numpy 中一种最基本的数组对象，其大小固定，**内部数据类型需要一致**，在数学运算的效率、稳定性上均有较大优势
- ② ndarray 复制：有三种形式，分别为引用、浅复制、深复制
- ③ ndarray 下标存取：与内置的 list 对象的存取和切片方法类似
- ④ ndarray 数组运算



认识 Python 标准库

□ numpy

numpy
以及众

相关概

①

②

③

④

1. #逐元素运算

2. $a + b$

3. $a * b$

4. a/b

5. $a ** 2$

6. $\text{np.sin}(a)$

7. #矩阵运算

8. $a.\text{dot}(b)$

9. $a = \text{np.mat}(a)$

10. $a.I$

11. $a.T$

12. $a.\text{trace}()$

13. $\text{np.linalg.det}(a)$

14. $\text{np.linalg.norm}(a, \text{ord}=\text{None})$

15. $\text{np.linalg.eig}(a)$

16. $\text{np.linalg.cond}(a, p=\text{None})$

#a 与 b 矩阵相乘

#转换为矩阵对象

#逆矩阵

#转置

#迹

#矩阵 a 的行列式

#矩阵 a 的范数

#矩阵 a 的特征值和特征向量

#矩阵 a 的条件数

对象,

内部



认识 Python 标准库

□ numpy

numpy 是一个用于**科学计算**的模块，提供了高性能的**数组对象**、**矩阵对象**，以及众多基础数学、线性代数、基本统计运算等所需的相关函数

相关概念

- ① ndarray 对象：numpy 中一种最基本的数组对象，其大小固定，**内部数据类型需要一致**，在数学运算的效率、稳定性上均有较大优势
- ② ndarray 复制：有三种形式，分别为引用、浅复制、深复制
- ③ ndarray 下标存取：与内置的 list 对象的存取和切片方法类似
- ④ ndarray 数组运算
- ⑤ ndarray 随机数生成

```
#生成均值为0，方差为1（不是标准差），长度为100的正态分布样本  
np.random.normal(0, 1, 100)
```



认识 Python 标准库

□ numpy

numpy 是一个用于**科学计算**的模块，提供了高性能的**数组对象**、**矩阵对象**，以及众多基础数学、线性代数、基本统计运算等所需的相关函数

相关概念

- ① ndarray 对象：numpy 中一种最基本的数组对象，其大小固定，**内部数据类型需要一致**，在数学运算的效率、稳定性上均有较大优势
- ② ndarray 复制：有三种形式，分别为引用、浅复制、深复制
- ③ ndarray 下标存取：与内置的 list 对象的存取和切片方法类似
- ④ ndarray 数组运算
- ⑤ ndarray 随机数生成
- ⑥



认识 Python 标准库

□ pandas

pandas 是一个**数据分析**模块，提供了快速、灵活、类似 SQL 中表的数据结构，能够便捷、高效地处理具有不同字段、不同来源的数据

相关概念

- ① DataFrame 数据类型：**类似于表的概念**，能够用标签对不同列进行标识



认识 Python 标准库

□ pandas

```

1. import pandas as pd
2. df = pd.DataFrame([[1, 2, 3], [4, 5, 6],
   [7, 8, 9]])
3. df.columns = ['A', 'B', 'C'] #定义表头
4. df['A']                      #取一列,
                               输出为

5. #0      1
6. #1      4
7. #2      7
8. df[['A', 'B']]              #取多列,
                               输出为

9. #AB
10. #0  1  2

11. #1  4  5
12. #2  7  8
13. df[1:3]                    #取1到2行,输出为
14. #A  B  C
15. #1  4  5  6
16. #2  7  8  9
17. df.iloc[1:3, 2]           #多维索引,输出为
18. #1      6
19. #2      9
20. #切片需要用 .iloc
21. df[df['A']>1]              #根据条件选择,输出为
22. #A  B  C
23. #1  4  5  6
24. #2  7  8  9

```

勾,

识



认识 Python 标准库

□ pandas

pandas 是一个**数据分析**模块，提供了快速、灵活、类似 SQL 中表的数据结构，能够便捷、高效地处理具有不同字段、不同来源的数据

相关概念

- ① DataFrame 数据类型：**类似于表的概念**，能够用标签对不同列进行标识
- ② DataFrame 的一列是一个 Series，每个 DataFrame 由多个 Series 组成

```
1. print(type(df.A))                #输出<class 'pandas.core.series.Series'>
2. tmp = pd.Series([1,2,3], name='tmp') #创建一个名为tmp的Series
```



认识 Python 标准库

□ pandas

pandas 是一个**数据分析**模块，提供了快速、灵活、类似 SQL 中表的数据结构，能够便捷、高效地处理具有不同字段、不同来源的数据

相关概念

- ① DataFrame 数据类型：**类似于表的概念**，能够用标签对不同列进行标识
- ② DataFrame 的一列是一个 Series，每个 DataFrame 由多个 Series 组成
- ③ 文件读取与写入

```
1. df = pd.read_csv('file.csv')           #读取 csv 格式文件,自动识别第一行为表头
2. df = pd.read_csv('file.csv', names=['A','B','C']) #指定表头
3. df = pd.read_csv('file.csv', index_col=0)      #指定索引列
4. df.to_csv('newfile.csv')                 #在工作目录下新建 csv 文件
```



认识 Python 标准库

□ pandas

pandas 是一个**数据分析**模块，提供了快速、灵活、类似 SQL 中表的数据结构，能够便捷、高效地处理具有不同字段、不同来源的数据

相关概念

- ① DataFrame 数据类型：**类似于表的概念**，能够用标签对不同列进行标识
- ② DataFrame 的一列是一个 Series，每个 DataFrame 由多个 Series 组成
- ③ 文件读取与写入
- ④ 队列操作

```
1. df['new'] = 1 #新建一列(添加为最后一列),赋值为1
2. df['new1'] = range(len(df)) #新建一列,赋值为由0开始的递增序列
3. df['date'] = ['20190801', '20190702', '20190601'] #新建一列存储日期
```



认识 Python 标准库

□ pandas

pandas 是一个**数据分析**模块，提供了快速、灵活、类似 SQL 中表的数据结构，能够便捷、高效地处理具有不同字段、不同来源的数据

相关概念

- ① DataFrame 数据类型：**类似于表的概念**，能够用标签对不同列进行标识
- ② DataFrame 的一列是一个 Series，每个 DataFrame 由多个 Series 组成
- ③ 文件读取与写入
- ④ 队列操作

```
#新建一列，赋值为列 A 与列 B 的和  
df[ 'new' ] = df[ 'A' ] + df[ 'B' ]
```

```
#新建一列，赋值为列 A 与列 B 的商，可以转换为 numpy 数组进行计算，提升性能  
df[ 'new' ] = df[ 'A' ].values / df[ 'B' ].values
```



认识 Python 标准库

□ pandas

pandas 是一个**数据分析**模块，提供了快速、灵活、类似 SQL 中表的数据结构，能够便捷、高效地处理具有不同字段、不同来源的数据

相关概念

- ① DataFrame 数据类型：**类似于表的概念**，能够用标签对不同列进行标识
- ② DataFrame 的一列是一个 Series，每个 DataFrame 由多个 Series 组成
- ③ 文件读取与写入
- ④ 队列操作

1. #任务:新建一列,将 date 字段的月份取出,并转换为 int 类型,此处使用了匿名函数,也可直接传入已有函数的函数名
2. `df['new'] = df['date'].apply(lambda x: int(x[4:6]))`
3. #任务:新建一列,对两列进行 apply 操作,获得列 A 与列 B 的和
4. `df['new'] = df[['A', 'B']].apply(lambda x: x[0]+x[1], axis=1)`



认识 Python 标准库

□ pandas

pandas 是一个**数据分析**模块，提供了快速、灵活、类似 SQL 中表的数据结构，能够便捷、高效地处理具有不同字段、不同来源的数据

相关概念

- ① DataFrame 数据类型：**类似于表的概念**，能够用标签对不同列进行标识
- ② DataFrame 的一列是一个 Series，每个 DataFrame 由多个 Series 组成
- ③ 文件读取与写入
- ④ 队列操作
- ⑤ 分组操作：将具有一个或多个相同字段的样本分为同一组，对这些组进行取平均值、最大值等操作

```
1.  #划分
2.  grouped = df.groupby(by=['A'])  #按列 A 进行分组
3.  grouped['C']                    #取分组后的 C 列
4.  df['C'].groupby(df['A'])         #等价于上面两行语句
```



认识 pandas

□ pandas

```
1. #划分
2. grouped = df.groupby(by=['A']) #按列 A 进行分组
3. grouped['C'] #取分组后的 C 列
4. df['C'].groupby(df['A']) #等价于上面两行语句
```

1. grouped.sum() #对各组进行单独求和
 2. grouped.size() #获取各组的大小
 3. grouped.mean() #获取各组的均值
 4. grouped.min() #获取各组最小值
 5. grouped.max() #获取各组最大值
 6. grouped.std() #获取各组标准差
 7. grouped.var() #获取各组方差
 8. #使用自定义函数
 9. grouped.aggregate(f) #f 为自定义函数 例如, np.mean, 也可采用匿名函数的形式

```
1. df = pd.DataFrame({'VehicleType': ['Car', 'Truck', 'Car', 'Truck'], 'Speed': [67., 43., 72., 49.]})
2. df.groupby(['VehicleType']).mean()
3. #输出
4. #VehicleType    Speed
5. #Car           69.5
6. #Truck          46.0
```

勾,

标识
组成



认识 Python 标准库

□ pandas

pandas 是一个**数据分析**模块，提供了快速、灵活、类似 SQL 中表的数据结构，能够便捷、高效地处理具有不同字段、不同来源的数据

相关概念

- ① DataFrame 数据类型：**类似于表的概念**，能够用标签对不同列进行标识
- ② DataFrame 的一列是一个 Series，每个 DataFrame 由多个 Series 组成
- ③ 文件读取与写入
- ④ 队列操作
- ⑤ 分组操作：将具有一个或多个相同字段的样本分为同一组，对这些组进行取平均值、最大值等操作
- ⑥ 数据拼接：主要有三种，分别为 append、concat、merge



认识 Python 标准库

□ pandas

⑥ 数据拼接：主要有三种，分别为 **append**、concat、merge

```
1. df = pd.DataFrame([[1, 2], [3, 4]], columns=list('AB'))
2. df2 = pd.DataFrame([[5, 6], [7, 8]], columns=list('AB'))
3. df.append(df2)
4. #输出
5. #   A  B
6. #0  1  2
7. #1  3  4
8. #0  5  6
9. #1  7  8
```



认识 Python 标准库

□ pandas

⑥ 数据拼接：主要有三种，分别为 `append`、**`concat`**、`merge`

```
1. df1 = pd.DataFrame([[ 'a', 1], [ 'b', 2]], columns=[ 'letter', 'number'])
2. df2 = pd.DataFrame([[ 'c', 3], [ 'd', 4]], columns=[ 'letter', 'number'])
3. pd.concat([df1, df2], axis=0)
4. #输出
5. #  letter  number
6. #0     a      1
7. #1     b      2
8. #0     c      3
9. #1     d      4
```

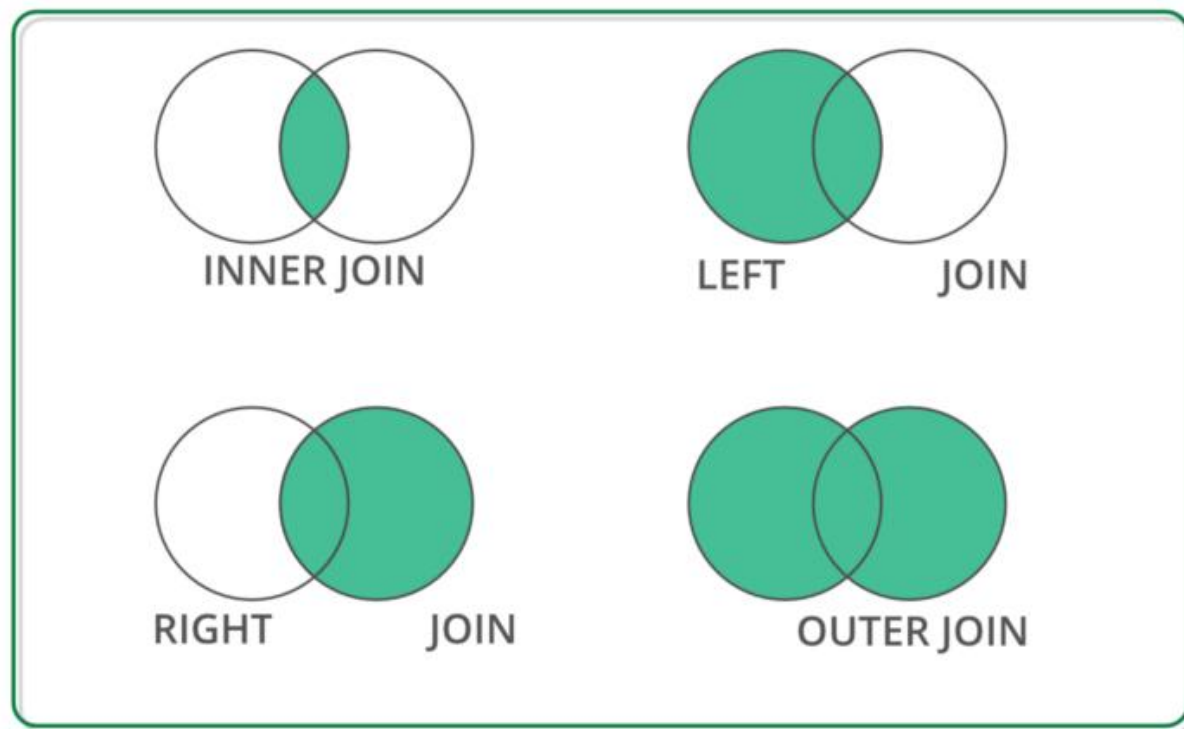


认识 Python 标准库

□ pandas

⑥ 数据拼接：主要有三种，分别为 append、concat、**merge**

merge 与 SQL 中表的连接方法类似，在应用中可自定义连接规则



认识 Python 标准库

□ pandas

pandas 是一个**数据分析**模块，提供了快速、灵活、类似 SQL 中表的数据结构，能够便捷、高效地处理具有不同字段、不同来源的数据

相关概念

- ① DataFrame 数据类型：**类似于表的概念**，能够用标签对不同列进行标识
- ② DataFrame 的一列是一个 Series，每个 DataFrame 由多个 Series 组成
- ③ 文件读取与写入
- ④ 队列操作
- ⑤ 分组操作：将具有一个或多个相同字段的样本分为同一组，对这些组进行取平均值、最大值等操作
- ⑥ 数据拼接：主要有三种，分别为 append、concat、merge
- ⑦ 统计特征函数：计算统计数据的均值、方差、标准差、分位数、相关系数、协方差等



认识 Python 标准库

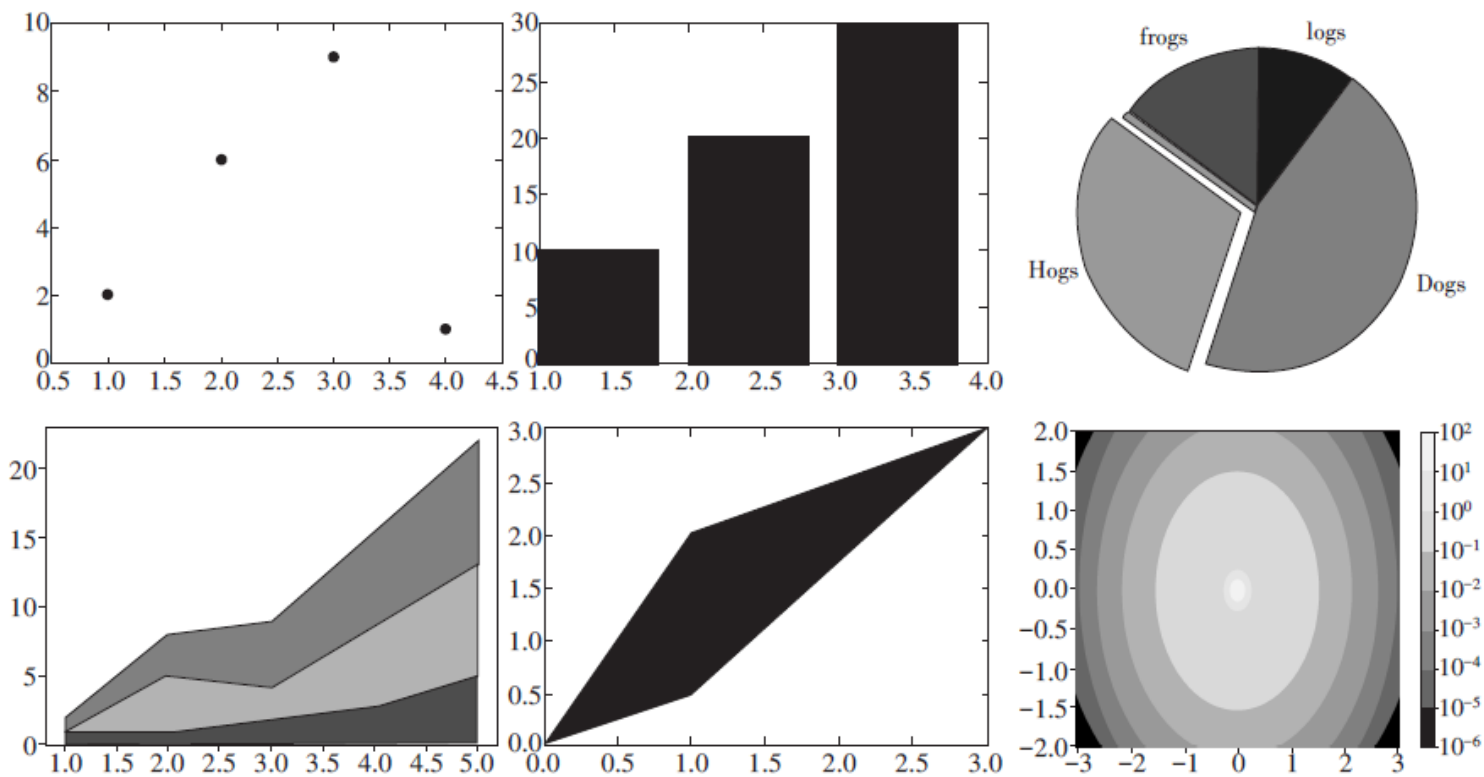
函数名	函数功能
<code>sum()</code>	数据样本的总和(按列计算)
<code>mean()</code>	数据样本的算术平均数
<code>median()</code>	数据样本的算术中位数(50%分位数)
<code>quantile()</code>	样本的分位数(0到1)
<code>min()</code> 、 <code>max()</code>	数据样本的最大值和最小值
<code>argmin()</code> 、 <code>argmax()</code>	最小值和最大值的索引位置(整数, Series 类型适用)
<code>var()</code>	数据样本的方差
<code>std()</code>	数据样本的标准差
<code>corr()</code>	数据样本的 Spearman(Pearson) 相关系数矩阵
<code>cov()</code>	数据样本的协方差矩阵
<code>diff()</code>	数据样本的一阶差分(对时间序列很有用)
<code>skew()</code>	样本值的偏度(三阶阵)
<code>kurt()</code>	样本值的峰度(四阶阵)
<code>describe()</code>	样本的基本描述(基本统计量如均值、标准差等)

认识 Python 标准库

□ matplotlib

matplotlib 是一个绘图工具，主要有以下几大功能：

- ① **基本绘图**：自带一些常用图形的绘制函数，例如散点图、条形图、直方图、堆积图、饼图、轮廓图等

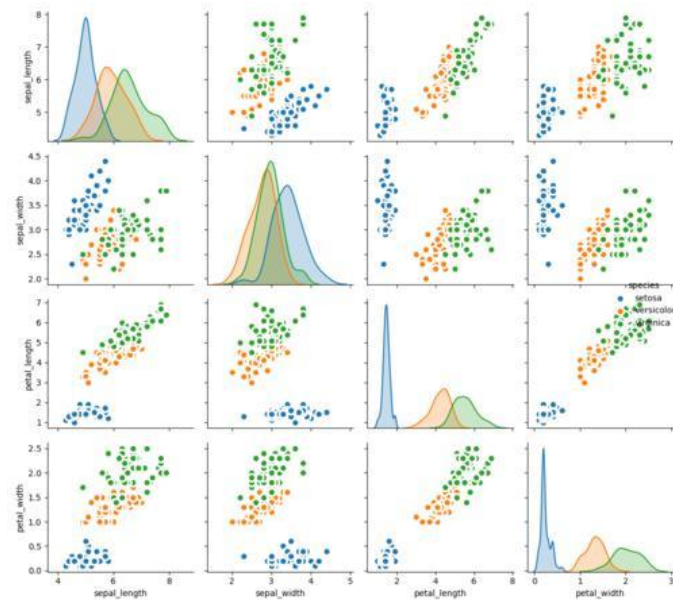
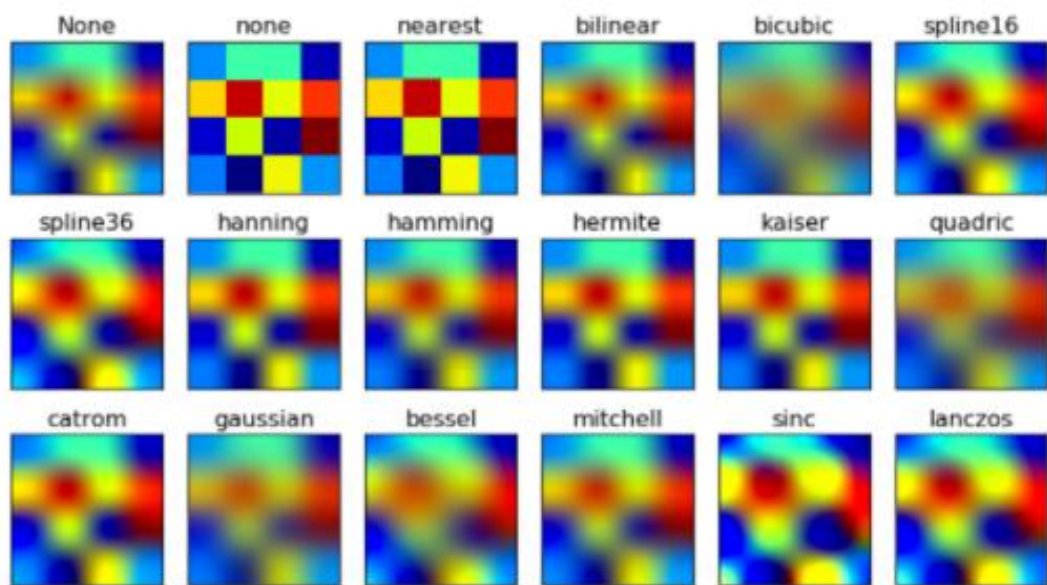


认识 Python 标准库

□ matplotlib

matplotlib 是一个绘图工具，主要有以下几大功能：

- ① **基本绘图**：自带一些常用图形的绘制函数，例如散点图、条形图、直方图、堆积图、饼图、轮廓图等
- ② **精细化绘图**：不仅能绘制基本的图表，还能基于各类参数对绘制图表的颜色、风格、坐标轴的范围和标签、图片的标注等进行深度定制和修改

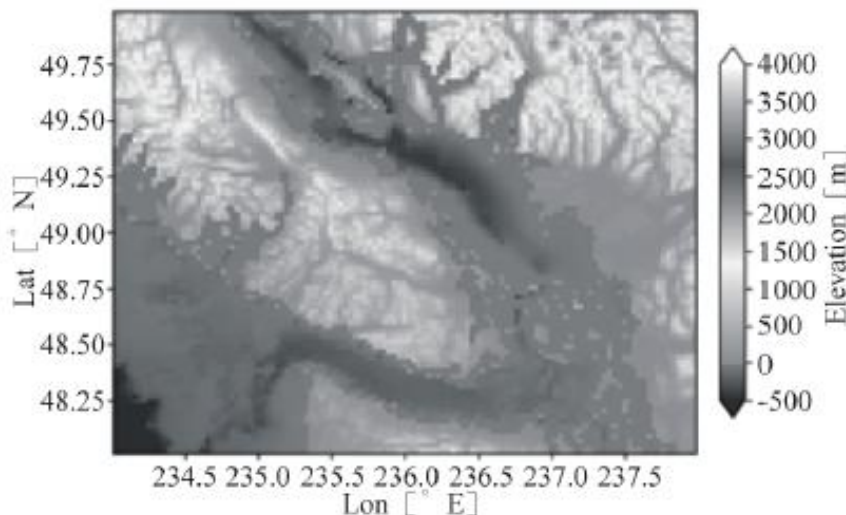
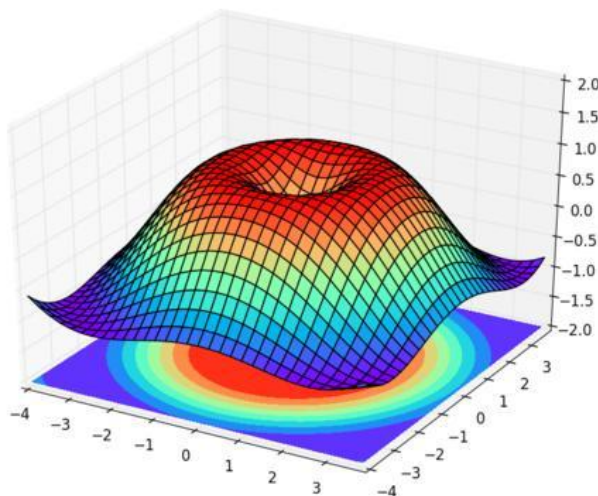


认识 Python 标准库

□ matplotlib

matplotlib 是一个绘图工具，主要有以下几大功能：

- ① **基本绘图**：自带一些常用图形的绘制函数，例如散点图、条形图、直方图、堆积图、饼图、轮廓图等
- ② **精细化绘图**：不仅能绘制基本的图表，还能基于各类参数对绘制图表的颜色、风格、坐标轴的范围和标签、图片的标注等进行深度定制和修改
- ③ **拓展应用**：其他高阶绘图功能，例如3D绘图、地理绘图等



<https://seaborn.pydata.org/>



认识 Python 标准库

□ scikit-learn

scikit-learn (sklearn) 是一个**机器学习**模块，封装了在构建机器学习模型时需要用到的相关功能，对各类模型的调用方法进行了统一化处理，减少了使用者的学习成本，能够快捷、方便地实现所需功能

- ① scikit-learn 提供了一些接口函数，能够直接进行**标准化、数据缩放、特征编码、缺失值填充、样本空间降维**等数据预处理
- ② scikit-learn 内置了许多常用模型：
 - **分类**：支持向量机、贝叶斯模型、决策树、集成学习等
 - **回归**：简单线性模型、广义线性模型、支持向量机回归、决策树回归、岭回归等
 - **聚类**：k-means、DBSCAN、层次聚类等
 - 模型参数标定、模型选择等



认识 Python 标准库

□ scikit-learn

```
1  import pandas as pd
2  import pprint
3  from sklearn.datasets import load_boston
4  from sklearn import linear_model
5  from sklearn.model_selection import train_test_split
6
7  boston = load_boston()
8  # pprint.pprint(boston)
9
10 df_x = pd.DataFrame(boston.data, columns = boston.feature_names)
11 df_y = pd.DataFrame(boston.target)
12
13 model = linear_model.LinearRegression()
14
15 x_train, x_test, y_train, y_test = train_test_split(df_x, df_y, test_size = 0.2, random_state = 4 )
16 model.fit(x_train,y_train)
17
18 result=model.predict(x_test)
19 print(result[5] , y_test )
20
```

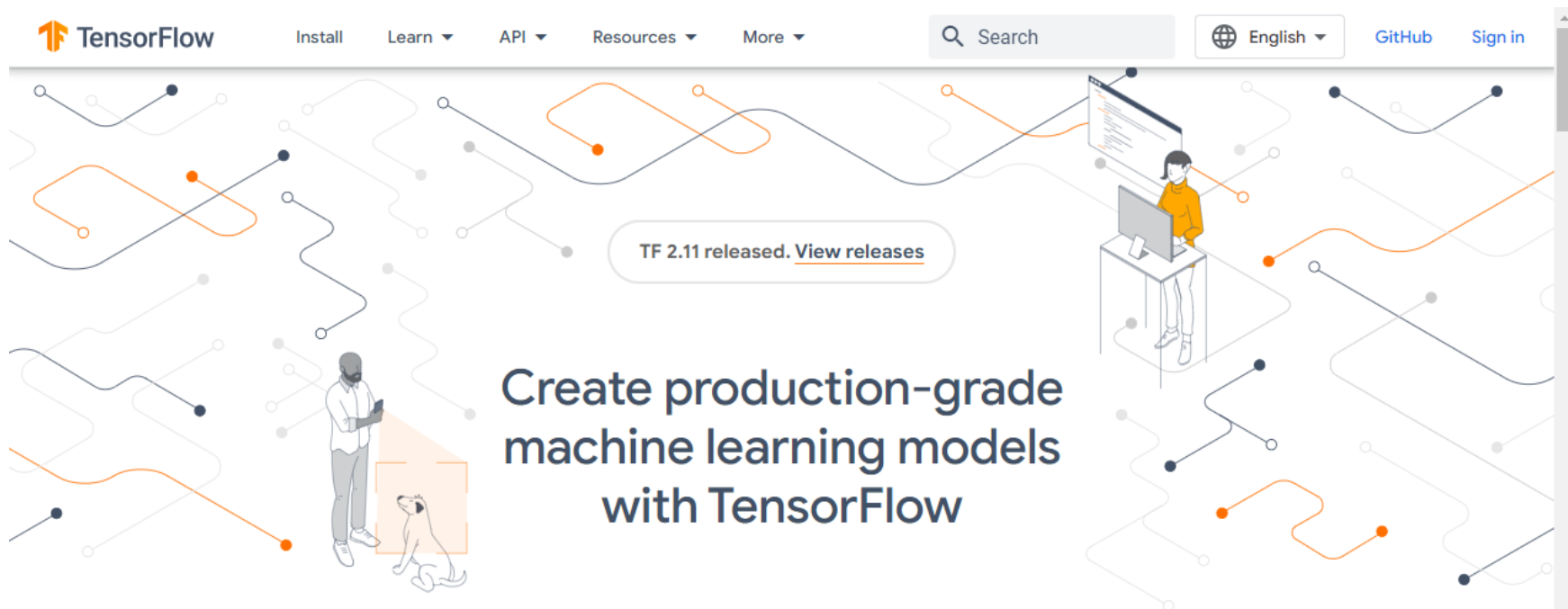


认识 Python 标准库

□ TensorFlow

TensorFlow 是**谷歌**开发的一个**开源机器学习**框架，提供了不同层级的模型接口，能实现简洁、快速的模型调用，也能借助底层接口实现高度定制化的机器学习模型构建

- ① TensorFlow 能够在 CPU 和 GPU 上运行，GPU能够极大提升神经网络的训练和推理速度，但对电脑的GPU具有一定的性能要求



认识 Python 标准库

□ TensorFlow

TensorFlow 是**谷歌**开发的一个**开源机器学习**框架，提供了不同层级的模型接口，能实现简洁、快速的模型调用，也能借助底层接口实现高度定制化的机器学习模型构建

② Colaboratory: 基于浏览器的编程环境（托管式Jupyter Notebook）

The screenshot shows the Google Colaboratory web interface. At the top, there's a header with the Google Colab logo, the text "Welcome To Colaboratory", and a menu bar with options: File, Edit, View, Insert, Runtime, Tools, Help. On the right of the header are links for "Share", "Settings", and a "Sign in" button. Below the header, the interface is divided into a left sidebar and a main content area. The sidebar contains a "Table of contents" section with a search icon and a list of links: "Getting started", "Data science", "Machine learning", "More Resources", and "Featured examples". The main content area displays a "Welcome to Colab!" message, followed by a paragraph: "If you're already familiar with Colab, check out this video to learn about interactive tables, the executed code history view, and the command palette." Below this text is a video thumbnail titled "3 Cool Google Colab Features" showing a man's face and the TensorFlow logo.

认识 Python 标准库

□ TensorFlow

TensorFlow 是**谷歌**开发的一个**开源机器学习**框架，提供了不同层级的模型接口，能实现简洁、快速的模型调用，也能借助底层接口实现高度定制化的机器学习模型构建

③ Keras: TensorFlow 高阶 API, 对 TensorFlow 底层 API 进行深度封装, 模型调用可概括为以下几个步骤:

- 模型定义
- 模型编译
- 模型训练
- 模型校验

深度学习模型一般由**前后“连接”的层**组成，只需定义一个模型，将所需的层添加到模型中，定义各层参数即可；模型定义之后，需要对模型进行编译，定义优化方法、损失函数、评价指标；最后进行模型训练和校验



认识

□ Tensor

TensorFlow
能实现
模型构建

③

深度学习

```

1. from tensorflow import keras
2. vocab_size = 10000                                #词汇表的大小,可理解为一个参数,
                                                    目前无需深究

3. #步骤一:模型定义
4. model = keras.Sequential()
5. model.add(keras.layers.Embedding(vocab_size, 16))    #添加 Embedding 层
6. model.add(keras.layers.GlobalAveragePooling1D())    #添加池化层
7. model.add(keras.layers.Dense(16, activation=tf.nn.relu)) #添加全连接层
8. model.add(keras.layers.Dense(1, activation=tf.nn.sigmoid)) #添加全连接层
9. model.summary()                                     #输出模型结构

10. #步骤二:模型编译
11. model.compile(optimizer='adam',                  #使用 adam 方法进行参数优化
12.                 loss='binary_crossentropy',      #损失函数选用二元交叉熵
13.                 metrics=['acc'])                 #评价指标采用准确率

14. x_val = trainData[:10000]                        #训练集中前 10000 个样本作为验证集
15. partial_x_train = trainData[10000:]
16. y_val = trainLabels[:10000]
17. partial_y_train = trainLabels[10000:]

18. #步骤三:模型训练
19. history = model.fit(partial_x_train,              #输入自变量
20.                     partial_y_train,              #输入因变量(标签)
21.                     epochs=40,                    #训练次数为 40
22.                     batch_size=512,               #分批训练,每批样本数为 512
23.                     validation_data=(x_val, y_val),
24.                     verbose=1)                    #显示每次训练的进度

25. #步骤四:模型校验
26. results = model.evaluate(testData, testLabels)
27. print(results)

```

模型接口,
机器学习模

深度封装,

将所需的
进行编译,



认识 Python 标准库

□ 本章小结

介绍了一种常用的高级编程语言 Python:

- ① 基础语法、特性、环境配置
- ② 模块安装和使用方法
- ③ 常用模块的功能和基本用法
- ④ TensorFlow等拓展性模块

Python 是一种高效的数据分析工具，可以帮助我们对数据进行处理和分析

在掌握 Python 基本知识的基础上，基于 Python 进行交通大数据分析还需要更多理论和方法层面的支撑！



认识 Python 标准库

□ 参考文献

- McKinney W. 利用Python进行数据分析 [M] . 北京: 机械工业出版社, 2018.
- UCI machine Learning Repository [EB/OL] [. 2020-06-10] .
- 廖雪峰. Python教程 [EB/OL] . [2020-06-10] .
- Python基础教程 [EB/OL] . [2020-06-10] .
- The Python Standard Library [EB/OL] . [2020-06-10] .
- Numpy [EB/OL] . [2020-06-30] . <https://www.numpy.org>.
- Vutukuru A. Different Types of Joins in SQL Server [EB/OL] . [2019-01-08] .
- 10Minutes to Pandas [EB/OL] . [2019-01-08] .
- Matplotlib Gallery [EB/OL] [. 2019-01-08] .
- Scikit-Learn User Guide [EB/OL] . [2019-01-08] .
- Text Classification with Preprocessed Text: Movie Reviews [EB/OL] . (2020-06-12) [2020-06-18] .

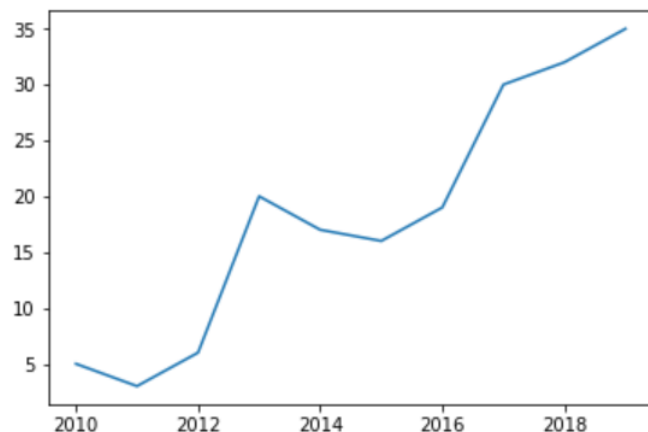


认识 Python 标准库

□ 练习

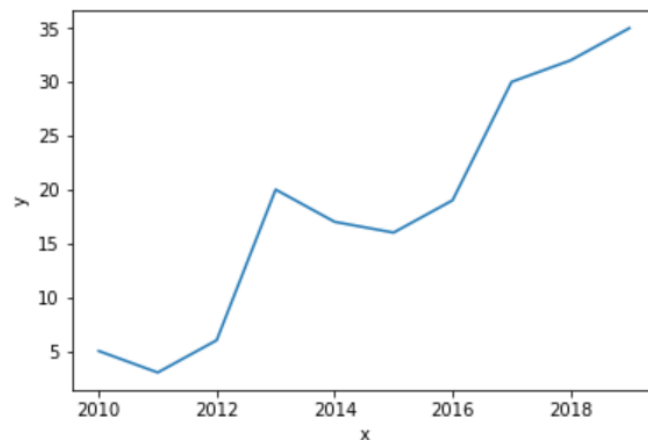
一、折线图

```
#import numpy as np  
#import pandas as pd  
#import matplotlib.pyplot as plt  
# import seaborn as sns
```



Matplotlib

```
x = [2010, 2011, 2012, 2013, 2014, 2015, 2016, 2017, 2018, 2019]  
y = [5, 3, 6, 20, 17, 16, 19, 30, 32, 35]  
plt.plot(x, y)
```



Seaborn

```
df = pd.DataFrame({'x': x, 'y': y})  
sns.lineplot(x="x", y="y", data=df)
```



认识 Python 标准库

□ 练习

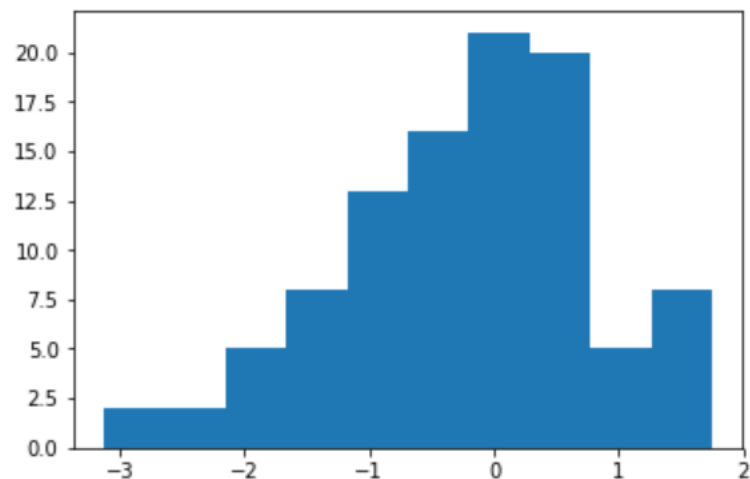
二、直方图

```
a = np.random.randn(100)
```

```
s = pd.Series(a)
```

Matplotlib

```
plt.hist(s)
```

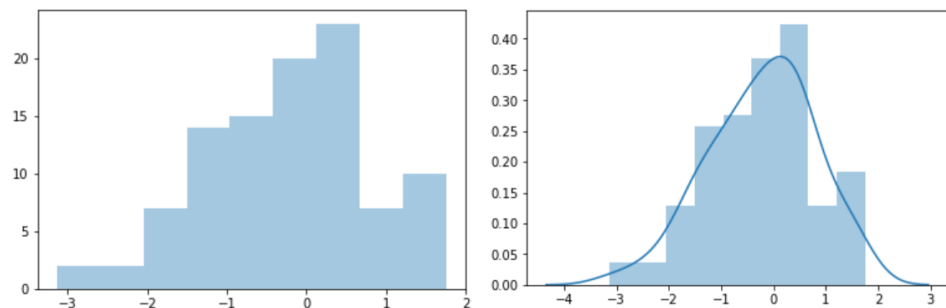


Seaborn

```
sns.distplot(s, kde=False)
```

```
plt.show()
```

```
sns.distplot(s, kde=True) plt.show()
```



认识 Python 标准库

□ 练习

三、垂直条形图

```
x = ['Cat1', 'Cat2', 'Cat3', 'Cat4', 'Cat5']
```

```
y = [5, 4, 8, 12, 7]
```

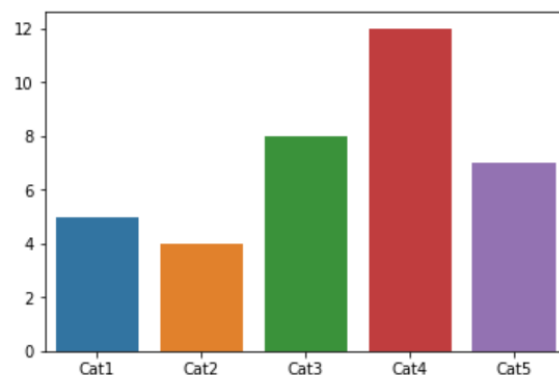
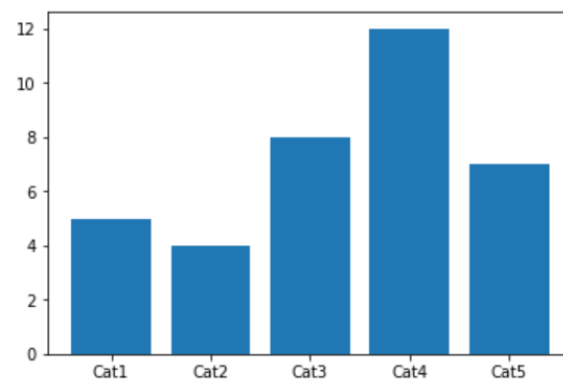
Matplotlib

```
plt.bar(x, y)
```

```
plt.show()
```

Seaborn

```
sns.barplot(x,y)
```



认识 Python 标准库

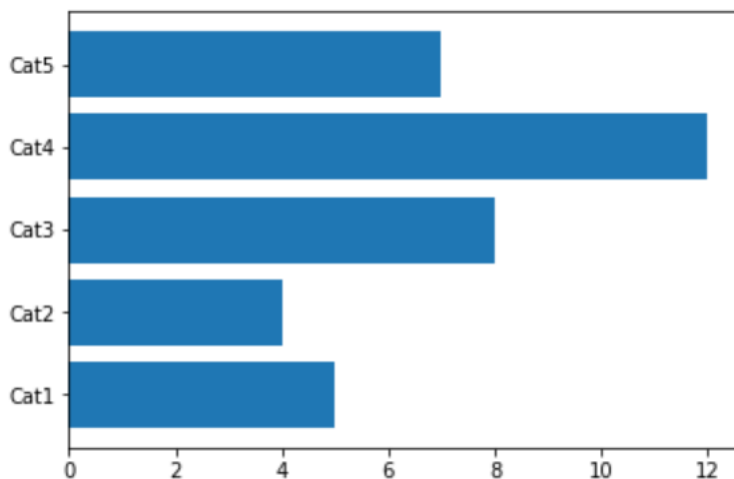
□ 练习

四、水平条形图

```
x = ['Cat1', 'Cat2', 'Cat3', 'Cat4', 'Cat5']
```

```
y = [5, 4, 8, 12, 7]
```

```
plt.barh(x, y)
```



认识 Python 标准库

□ 练习

五、饼图

```
nums = [25, 37, 33, 37, 6]
```

```
labels = ['High-school', 'Bachelor', 'Master', 'Ph.d', 'Others']
```

```
plt.pie(x = nums, labels=labels)
```

```
plt.show()
```



认识 Python 标准库

□ 练习

六、箱线图

```
data=np.random.normal(size=(10,4))
```

```
lables = ['A','B','C','D']
```

Matplotlib

```
plt.boxplot(data,labels=lables)
```

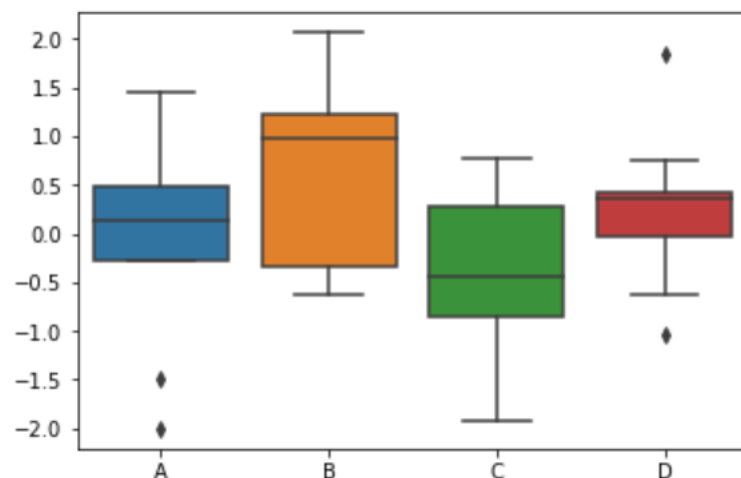
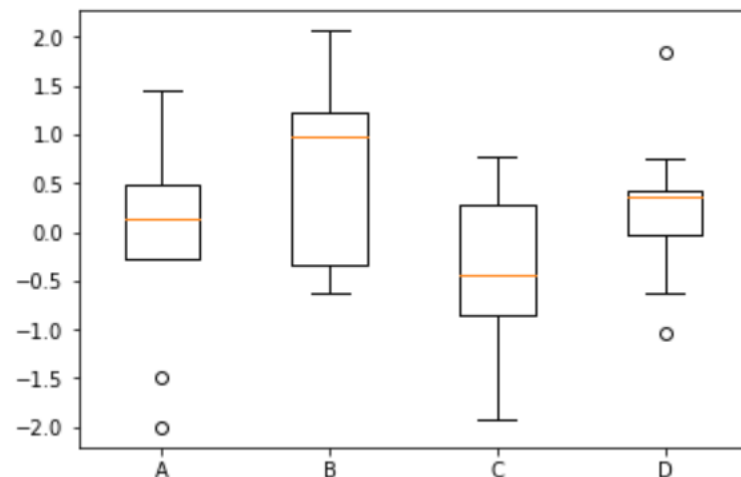
```
plt.show()
```

Seaborn

```
df = pd.DataFrame(data, columns=lables)
```

```
sns.boxplot(data=df)
```

```
plt.show()
```



认识 Python 标准库

□ 练习

七、散点图

$N = 1000$

```
x = np.random.randn(N)
```

```
y = np.random.randn(N)
```

Matplotlib

```
plt.scatter(x, y, marker='x')
```

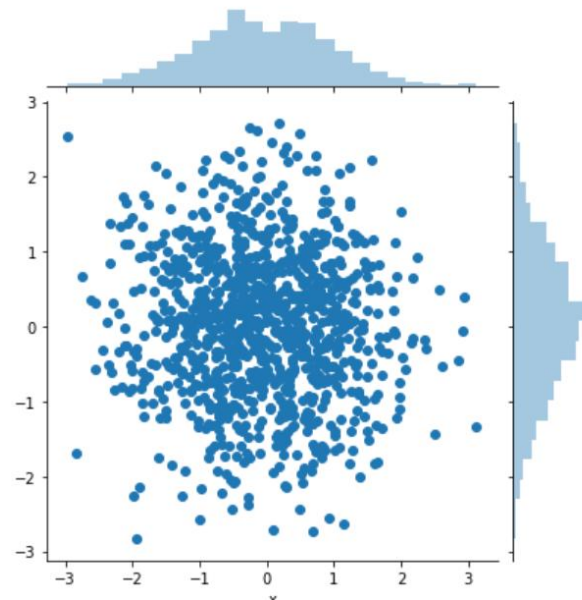
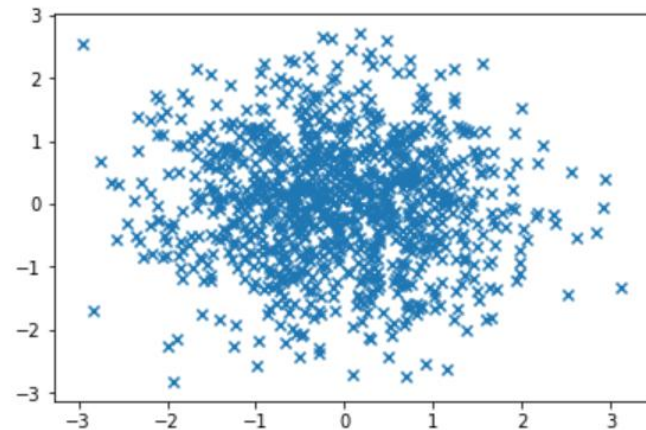
```
plt.show()
```

Seaborn

```
df = pd.DataFrame({'x': x, 'y': y})
```

```
sns.jointplot(x="x", y="y", data=df, kind='scatter');
```

```
plt.show()
```



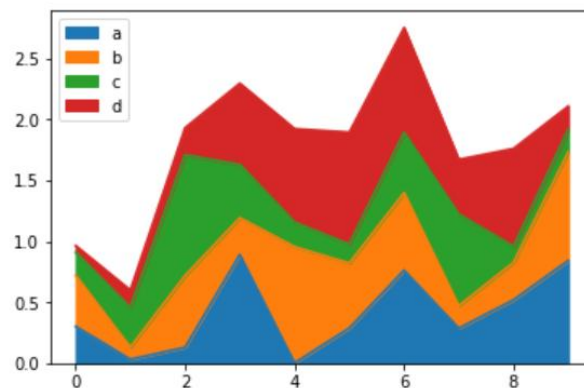
认识 Python 标准库

□ 练习

八、面积图

```
df = pd.DataFrame(np.random.rand(10, 4), columns=['a', 'b', 'c', 'd'])
```

```
df.plot.area()
```



```
df.plot.area(stacked=False)
```

